

Week 4: Local Database and Local Storage

1. Save data in a local database using Room
 - a. Room
 - b. Primary components
 - c. Define data using Room entities
 - i. @Entity
 - ii. @PrimaryKey
 - iii. @ColumnInfo
 - iv. @Ignore
 - d. Accessing data using Room DAOs
 - i. Convenience methods
 1. @Insert
 2. @Update
 3. @Upsert
 4. @Delete
 - ii. Query methods
 1. @Query
 - e. Database
 - f. Usage
2. Define relationships between objects
 - a. Create embedded objects
 - i. @Embedded

- b. Define one-to-one relationships
 - i. @Relation
 - ii. @Transaction
 - c. Define one-to-many relationships
 - d. Define many-to-many relationships
3. Write asynchronous DAO queries
 - a. Write asynchronous one-shot queries
 - b. Write observable queries
 - i. LiveData
 - ii. Flow
4. Migrate your Room database
 - a. Automated migrations
 - i. Automatic migration specifications
 1. @DeleteTable
 2. @RenameTable
 3. @DeleteColumn
 4. @RenameColumn
 - b. Manual migrations
5. Data and file storage overview
 - a. App-specific storage
 - b. Shared storage
 - c. Preferences
 - d. Database
6. App specific storage
 - a. Internal storage
 - b. External storage
 - c. Access from internal storage
 - i. Access persistent files

- ii. Access cache files
- d. Access from external storage
 - i. Access persistent files
 - ii. Access cache files

7. Shared storage

- a. Media content
- b. Documents and other files
- c. Datasets
- d. Access media files from shared storage
 - i. Media store
 - ii. Access other apps' media files

Save data in a local database using Room

The Room persistence library provides an abstraction layer over SQLite to allow fluent database access.

Room provides the following benefits:

- Compile-time verification of SQL queries.
- Convenience annotations that minimize repetitive and error-prone boilerplate code.
- Streamlined database migration paths.

Primary components

There are three major components in Room:

- The **database class** that holds the database and serves as the main access point for the underlying connection to your app's persisted data.
- **Data entities** that represent tables in your app's database.
- **Data access objects (DAOs)** that provide methods that your app can use to query, update, insert, and delete data in the database.

Define data using Room entities

Each entity corresponds to a table in the associated Room database, and each instance of an entity represents a row of data in the corresponding table.

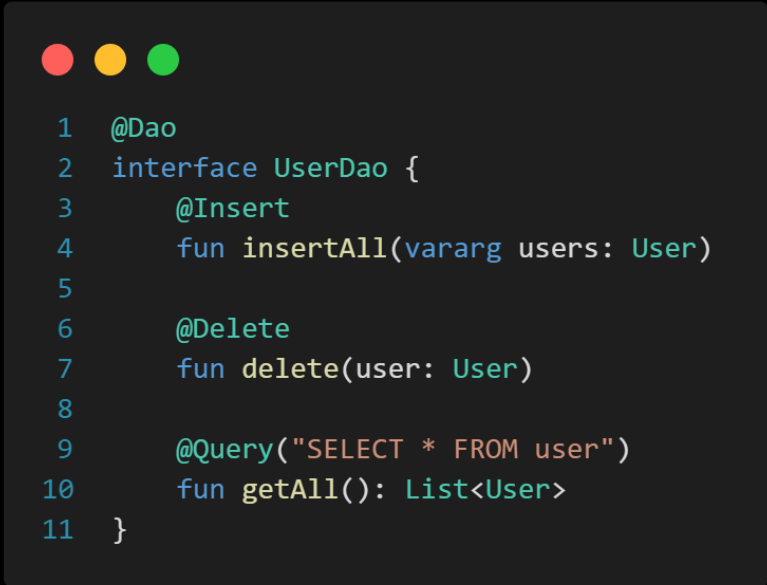


```
1  @Entity(tableName = "users")
2  data class User (
3      @PrimaryKey val id: Int,
4      @ColumnInfo(name = "first_name") val firstName: String?,
5      @ColumnInfo(name = "last_name") val lastName: String?
6  )
```

- **@Entity:** You define each Room entity as a class annotated with @Entity.
- **@PrimaryKey:** Each Room entity must define a primary key that uniquely identifies each row in the corresponding database table.
- **@ColumnInfo:** Used to customize the column.
- **@Ignore:** By default, Room creates a column for each field that's defined in the entity. If an entity has fields that you don't want to persist, you can annotate them using @Ignore.

Accessing data using Room DAOs

You interact with the stored data by defining data access objects, or DAOs. Each DAO includes methods that offer abstract access to your app's database. At compile time, Room automatically generates implementations of the DAOs that you define.



```
1  @Dao
2  interface UserDao {
3      @Insert
4      fun insertAll(vararg users: User)
5
6      @Delete
7      fun delete(user: User)
8
9      @Query("SELECT * FROM user")
10     fun getAll(): List<User>
11 }
```

There are two types of DAO methods that define database interactions:

- Convenience methods that let you insert, update, and delete rows in your database without writing any SQL code.
- Query methods that let you write your own SQL query to interact with the database.

Convenience methods

- **@Insert:** Insert into the table.
- **@Update:** Update rows in the table.
- **@Upsert:** Similar to `@Insert(onConflict = OnConflictStrategy.REPLACE)`, it update the value if the corresponding row already exists in the table otherwise insert the value.
- **@Delete:** delete specific rows from a database table.

Query methods

The @Query annotation lets you write SQL statements and expose them as DAO methods.

@Query("SELECT * FROM user")	This query selects all fields (*) from the user table, fetching all the user records.
@Query("SELECT first_name, last_name FROM user")	This query selects only the first_name and last_name columns from the user table, fetching just these two fields for each user.
@Query("SELECT * FROM user WHERE age > :minAge")	This query selects all fields from the user table for users whose age is greater than the parameter minAge.
@Query("SELECT * FROM user WHERE age BETWEEN :minAge AND :maxAge")	This query selects all fields from the user table for users whose age falls between the specified minAge and maxAge.
@Query("SELECT * FROM user WHERE first_name LIKE :search OR last_name LIKE :search")	This query selects all fields from the user table for users where either the first_name or last_name contains a string that matches the search parameter (typically used for partial matches).
@Query("SELECT * FROM user WHERE region IN (:regions)")	This query selects all fields from the user table for users whose region matches any of the values in the regions parameter, which is a collection (e.g., a list of regions).

@Query("SELECT * FROM user WHERE age > :minAge LIMIT 5")	This query selects all fields from the user table for users older than the specified minAge, but limits the results to only 5 records.
@Query("SELECT * FROM users WHERE label LIKE :query")	This query selects all fields from the users table where the label column contains a string that matches the query parameter (useful for partial string matches).

Database



```
1 @Database(entities = [User::class], version = 1)
2 abstract class AppDatabase : RoomDatabase() {
3     abstract fun userDao(): UserDao
4 }
```

Usage



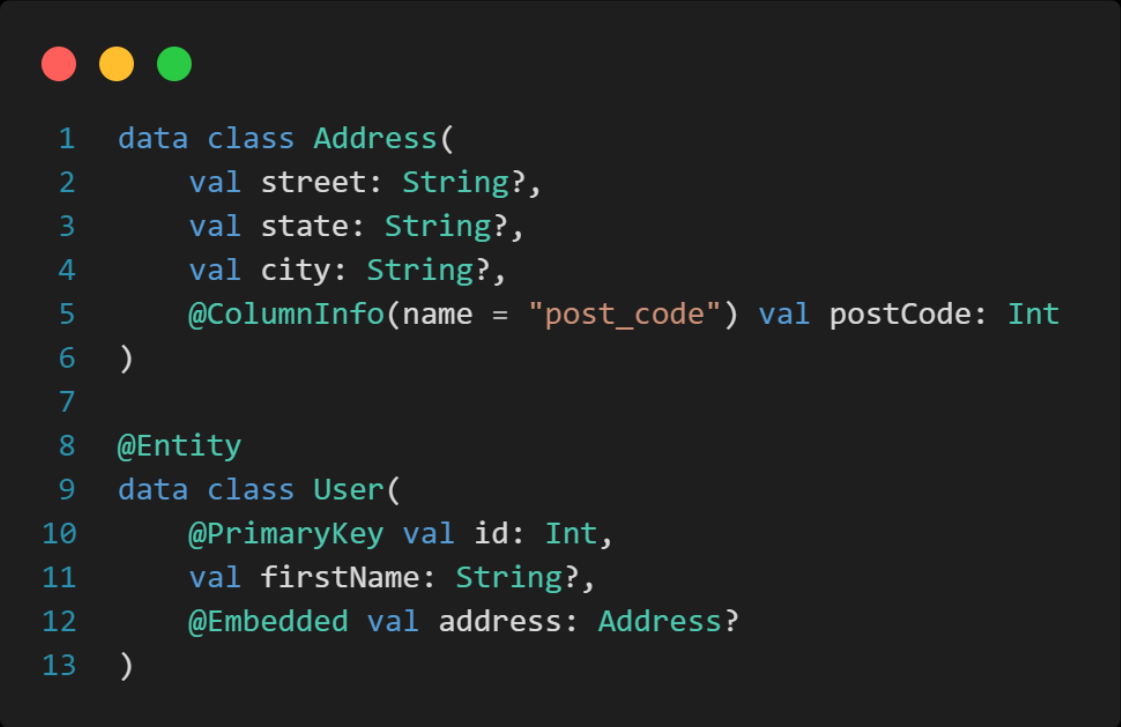
```
1 val db = Room.databaseBuilder(
2     applicationContext,
3     AppDatabase::class.java, "database-name"
4 ).build()
5
6 val userDao = db.userDao()
7 val users: List<User> = userDao.getAll()
```

Define relationships between objects

In Room, there are two ways to define and query a relationship between entities: using either an intermediate data class with embedded objects or a relational query method with a multimap return type.

Create embedded objects

It allows you to include the fields of another class (which itself is not an entity) directly in your Room database table.



```
1  data class Address(  
2      val street: String?,  
3      val state: String?,  
4      val city: String?,  
5      @ColumnInfo(name = "post_code") val postCode: Int  
6  )  
7  
8  @Entity  
9  data class User(  
10     @PrimaryKey val id: Int,  
11     val firstName: String?,  
12     @Embedded val address: Address?  
13 )
```

The table representing a User object then contains columns with the following names: id, firstName, street, state, city, and post_code.

Define one-to-one relationships

A one-to-one relationship means that each instance of an entity corresponds to exactly one instance of another entity and vice versa.

To define a one-to-one relationship, first create a class for each of your two entities. One of the entities must include a variable that is a reference to the primary key of the other entity.



```
1  @Entity
2  data class User(
3      @PrimaryKey val userId: Long,
4      val name: String,
5      val age: Int
6  )
7
8  @Entity
9  data class Library(
10     @PrimaryKey val libraryId: Long,
11     val userOwnerId: Long
12 )
13
14 data class UserAndLibrary(
15     @Embedded val user: User,
16     @Relation(
17         parentColumn = "userId",
18         entityColumn = "userOwnerId"
19     )
20     val library: Library
21 )
22
23 @Transaction
24 @Query("SELECT * FROM User")
25 fun getUsersAndLibraries(): List<UserAndLibrary>
```

- @Embedded: This annotation is used to include all fields of the User object directly inside the UserAndLibrary class.

- **@Relation:** This annotation establishes the relationship between User and Library.
 - **parentColumn = "userId":** This indicates that the primary key in the User table is userId.
 - **entityColumn = "userOwnerId":** This indicates that the Library table has a foreign key column userOwnerId, which refers to userId in the User table.
- **@Transaction:** Ensures that the operation is performed atomically. Since Room needs to execute two queries (one to fetch User and another to fetch the associated Library), marking the method with @Transaction ensures that both queries are completed as a single unit.
- **@Query("SELECT * FROM User"):** This query retrieves all the users, and Room will automatically pair each User with its corresponding Library by using the UserAndLibrary data class.

Define one-to-many relationships

A one-to-many relationship means that each instance of the parent entity can correspond to multiple instances of the child entity. However, each child entity instance corresponds to exactly one parent.

Define many-to-many relationships

A many-to-many relationship means that each instance of the parent entity can correspond to multiple instances of the child entity, and each instance of the child entity can also correspond to multiple instances of the parent entity.

Write asynchronous DAO queries

DAO queries fall into three categories:

- One-shot write queries that insert, update, or delete data in the database.
- One-shot read queries that read data from your database only once and return a result with the snapshot of the database at that time.
- Observable read queries that read data from your database every time the underlying database tables change and emit new values to reflect those changes.

Write asynchronous one-shot queries

One-shot queries are database operations that only run once and grab a snapshot of data at the time of execution.

```
1  @Dao
2  interface UserDao {
3      @Insert(onConflict = OnConflictStrategy.REPLACE)
4      suspend fun insertUsers(vararg users: User)
5
6      @Update
7      suspend fun updateUsers(vararg users: User)
8
9      @Delete
10     suspend fun deleteUsers(vararg users: User)
11
12     @Query("SELECT * FROM user WHERE id = :id")
13     suspend fun loadUserById(id: Int): User
14
15     @Query("SELECT * from user WHERE region IN (:regions)")
16     suspend fun loadUsersByRegion(regions: List<String>): List<User>
17 }
```

Write observable queries

Observable queries are read operations that emit new values whenever there are changes to any of the tables that are referenced by the query.



```
1 @Dao
2 interface UserDao {
3     @Query("SELECT * FROM user WHERE id = :id")
4     fun loadUserById(id: Int): Flow<User>
5
6     @Query("SELECT * from user WHERE region IN (:regions)")
7     fun loadUsersByRegion(regions: List<String>): Flow<List<User>>
8 }
```

Migrate your Room database

Room supports both automated and manual options for incremental migration. Automatic migrations work for most basic schema changes, but you might need to manually define migration paths for more complex changes.

Automated migrations



```
1  @Database(  
2      version = 2,  
3      entities = [User::class],  
4      autoMigrations = [  
5          AutoMigration (from = 1, to = 2)  
6      ]  
7  )  
8  abstract class AppDatabase : RoomDatabase() {  
9      ...  
10 }
```

For auto migration you must set a schemaDirectory in the gradle



```
1  plugins {
2      id("androidx.room")
3  }
4
5  android {
6      ...
7      room {
8          schemaDirectory("$projectDir/schemas")
9      }
10 }
11
12 // OR
13
14 android {
15     ...
16     defaultConfig {
17         ...
18         ksp {
19             arg("room.schemaLocation", "$projectDir/schemas")
20         }
21     }
22 }
```

Automatic migration specifications

- Deleting or renaming a table.
- Deleting or renaming a column.

You can use `AutoMigrationSpec` to give Room the additional information that it needs to correctly generate migration paths. Define a static class that implements `AutoMigrationSpec` in your `RoomDatabase` class and annotate it with one or more of the following:

- `@DeleteTable`
- `@RenameTable`
- `@DeleteColumn`
- `@RenameColumn`



```

1  @Database(
2      version = 2,
3      entities = [User::class],
4      autoMigrations = [
5          AutoMigration (
6              from = 1,
7              to = 2,
8              spec = AppDatabase.MyAutoMigration::class
9          )
10     ]
11 )
12 abstract class AppDatabase : RoomDatabase() {
13     @RenameTable(fromTableName = "User", toTableName = "AppUser")
14     class MyAutoMigration : AutoMigrationSpec
15     ...
16 }

```

Manual migrations

In cases where a migration involves complex schema changes



```

1  val MIGRATION_1_2 = object : Migration(1, 2) {
2      override fun migrate(database: SupportSQLiteDatabase) {
3          database.execSQL("CREATE TABLE `Fruit` (`id` INTEGER, `name` TEXT, " +
4              "PRIMARY KEY(`id`))")
5      }
6  }
7
8  val MIGRATION_2_3 = object : Migration(2, 3) {
9      override fun migrate(database: SupportSQLiteDatabase) {
10         database.execSQL("ALTER TABLE Book ADD COLUMN pub_year INTEGER")
11     }
12 }
13
14 Room.databaseBuilder(applicationContext, MyDb::class.java, "database-name")
15     .addMigrations(MIGRATION_1_2, MIGRATION_2_3).build()

```

Data and file storage overview

The system provides several options for you to save your app data:

- **App-specific storage:** Store files that are meant for your app's use only, either in dedicated directories within an internal storage volume or different dedicated directories within external storage. Use the directories within internal storage to save sensitive information that other apps shouldn't access.
- **Shared storage:** Store files that your app intends to share with other apps, including media, documents, and other files.
- **Preferences:** Store private, primitive data in key-value pairs.
- **Databases:** Store structured data in a private database using the Room persistence library.

	Type of content	Access method	Permissions needed	Can other apps access?	Files removed on app uninstall?
App-specific files	Files meant for your app's use only	From internal storage, <code>getFilesDir()</code> or <code>getCacheDir()</code> From external storage, <code>getExternalFilesDir()</code> or <code>getExternalCacheDir()</code>	Never needed for internal storage Not needed for external storage when your app is used on devices that run Android 4.4 (API level 19) or higher	No	Yes
Media	Shareable media files (images, audio files, videos)	MediaStore API	<code>READ_EXTERNAL_STORAGE</code> when accessing other apps' files on Android 11 (API level 30) or higher <code>READ_EXTERNAL_STORAGE</code> or <code>WRITE_EXTERNAL_STORAGE</code> when accessing other apps' files on Android 10 (API level 29) Permissions are required for all files on Android 9 (API level 28) or lower	Yes, though the other app needs the <code>READ_EXTERNAL_STORAGE</code> permission	No
Documents and other files	Other types of shareable content, including downloaded files	Storage Access Framework	None	Yes, through the system file picker	No
App preferences	Key-value pairs	Jetpack Preferences library	None	No	Yes
Database	Structured data	Room persistence library	None	No	Yes

App specific storage

- Files saved in app-specific storage are removed when the app is uninstalled.
- This storage is suitable for data that other apps shouldn't access.

Internal storage

- used for persistent and cached files.
- It doesn't require system permissions.
- Not accessible by other apps.
- Used to store sensitive data.
- Deleted when app uninstalled.
- Encrypted on Android 10 and higher.

External storage

- Used for persistent and cached files.
- It doesn't require system permissions.
- Accessible by other apps with proper permission (Until Android 11)
- Do not store sensitive data.
- Deleted when app uninstalled.

Access from internal storage

Access persistent files

- Access using the *filesDir* property of a context object.



```
1 val file = File(context.filesDir, filename)
2 // Store a file
3 file.writeText("Hello world!")
4 // Access a file
5 file.readText()
```



```
1 val fileContents = "Hello world!"
2
3 // Store using stream
4 context.openFileOutput(filename, Context.MODE_PRIVATE).use {
5     it.write(fileContents.toByteArray())
6 }
7
8 // Access a file using a stream
9 context.openFileInput(filename).bufferedReader().useLines { lines ->
10     lines.fold("") { some, text ->
11         "$some\n$text"
12     }
13 }
```

- You can use the *File* API to access and store files.
- As an alternative to using the *File* API, you can call *openFileOutput()* to get a *FileOutputStream* that writes to a file within the *filesDir* directory.
- To read a file as a stream, use *openFileInput()*



```
1 // View list of files
2 var files: Array<String> = context.fileList()
3
4 // Create nested directories
5 context.getDir(dirName, Context.MODE_PRIVATE)
```

- filesDir is always an ancestor directory of this new directory.

Access cache files



```
1 // To create a cached file
2 File.createTempFile(filename, null, context.cacheDir)
3
4 // To access a cached file
5 val cacheFile = File(context.cacheDir, filename)
6
7 // Remove cache files
8
9 cacheFile.delete()
10
11 context.deleteFile(cacheFileName)
```

- When the device is low on internal storage space, Android may delete these cache files to recover space. So check for the existence of your cache files before reading them.

Access from external storage

- Because external storage resides on a physical volume that the user might be able to remove, verify that the volume is accessible before

trying to read app-specific data from, or write app-specific data to, external storage.

```
1 // Checks if a volume containing external storage is available
2 // for read and write.
3 fun isExternalStorageWritable(): Boolean {
4     return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED
5 }
6
7 // Checks if a volume containing external storage is available to at least read.
8 fun isExternalStorageReadable(): Boolean {
9     return Environment.getExternalStorageState() in
10         setOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)
11 }
```

Access persistent files

- To access app-specific files from external storage, call *getExternalFilesDir()*.

```
1 val appSpecificExternalDir = File(context.getExternalFilesDir(null), filename)
```

Access cache files

```
1 // Create cache files
2 val externalCacheFile = File(context.externalCacheDir, filename)
3
4 // Remove cache files
5 externalCacheFile.delete()
```

Shared storage

- Use shared storage for user data that can or should be accessible to other apps and saved even if the user uninstalls your app.
- Android provides APIs for storing and accessing the following types of shareable data:
 - I. **Media content:** The system provides standard public directories for these kinds of files, so the user has a common location for all their photos, another common location for all their music and audio files, and so on. Your app can access this content using the platform's *MediaStore* API.
 - II. **Documents and other files:** The system has a special directory for containing other file types, such as PDF documents and books that use the EPUB format. Your app can access these files using the platform's Storage Access Framework.
 - III. **Datasets:** On Android 11 (API level 30) and higher, the system caches large datasets that multiple apps might use. These datasets can support use cases like machine learning and media playback. Apps can access these shared datasets using the *BlobStoreManager* API.

Access media files from shared storage

Media store

To interact with the media store abstraction, use a `ContentResolver` object that you retrieve from your app's context:

```

1  applicationContext.contentResolver
2      .query(
3          /* uri = */ MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
4          /* projection = */ null,
5          /* selection = */ null,
6          /* selectionArgs = */ null,
7          /* sortOrder = */ null,
8     )?.use { cursor ->
9          val idColumn = cursor.getColumnIndexOrThrow(MediaStore.Images.Media._ID)
10
11          while (cursor.moveToNext()) {
12              val id = cursor.getLong(idColumn)
13              val contentUri: Uri =
14                  ContentUris.withAppendedId(
15                      MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
16                      id,
17                  )
18          }
19      }

```

The system automatically scans an external storage volume and adds media files to the following well-defined collections:

- **Images** (MediaStore.Images) - DCIM/ and Pictures/.
- **Videos** (MediaStore.Video) - DCIM/, Movies/, and Pictures/.
- **Audio files** (MediaStore.Audio) - Alarms/, Audiobooks/, Music/, Notifications/, Podcasts/, and Ringtones/.

Additionally Music/ or Movies/.

Recordings/ (Not available on Android 11 (API level 30) and lower)

- **Downloaded files** (MediaStore.Downloads) - Download/ (Not available on Android 9 and lower)
- (MediaStore.Files) - If scoped storage is unavailable or not being used, the collection shows all types of media files.
If scoped storage is enabled, the collection shows only the photos, videos, and audio files that your app has created.

Access other apps' media files

To access media files that other apps create, you must declare the appropriate storage-related permissions

```
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"  
/>
```

Shared storage

- Use shared storage for user data that can or should be accessible to other apps and saved even if the user uninstalls your app.
 - Media content: Your app can access this content using the platform's MediaStore API.
 - Documents and other files: Your app can access these files using the platform's Storage Access Framework.
 - Datasets: Apps can access these shared datasets using the BlobStoreManager API.

Scoped storage

- Introduced in Android 10
- Scoped storage limits app access to external storage.
- Files accessed via MediaStore API and Storage Access Framework.

Storage Access Framework (SAF)

- The SAF lets users browse and open documents, images, and other files across all of their preferred document storage providers.
- Allow users to select files via a system file picker, improving user control and privacy.

BlobStoreManager

- On Android 11 (API level 30) and higher, the system caches large datasets that multiple apps might use.
- Apps can access these shared datasets using the BlobStoreManager API.

Content providers

- Content providers can help an application manage access to data stored by itself or stored by other apps and provide a way to share data with other apps.

File providers

- FileProvider is a special subclass of ContentProvider that facilitates secure sharing of files associated with an app by creating a content://Uri for a file instead of a file:///Uri.
- A special type of content provider that securely grants access to files in an app's private storage to other apps by generating URIs without exposing the actual file paths.
- Allows apps to share files while keeping them secure and private, allowing developers to grant temporary read and write access.

Content resolver

- Class that provides applications access to the content model.
- Acts as an intermediary to access data from content providers, enabling apps to query, insert, update, or delete shared data from other apps or system content like contacts or media.
- Provides access to content models and allows apps to interact with content from file systems and installed apps.

Cursor

- This interface provides random read-write access to the result set returned by a database query.

InputStream

- The InputStream class is used for reading data from a source (such as a file, network connection, or memory).
- It reads data as a sequence of bytes (binary data).

OutputStream

- The OutputStream class is used for writing data to a destination (such as a file, network socket, or memory).
- It writes data as a sequence of bytes.

FileInputStream

- A class that allows reading raw binary data from a file, typically used for reading files stored on the device.

FileOutputStream

- A class that provides a way to write binary data (like images or files) directly to a file in the app's internal or external storage.

BufferedReader

- A class used for efficiently reading text from an input stream, such as files, by buffering the input to minimize interaction with the underlying file system.

openInputStream()

- It opens an input stream for reading data from a content URI.
- You use this method when you want to read data from a content URI (e.g., to read a file or an image).

openOutputStream()

- It opens an output stream for writing data to a content URI.
- You use this method when you want to write data to a content URI (e.g., saving a file or an image to a content provider).